

DMS 文件管理系統 V6 - 系統功能規格說明書

本文件詳列「文件管理系統 V6 (DMS V6)」之系統規格、架構設計、核心業務邏輯、API 與資料庫對接機制、狀態管理與版本控管規劃。

1. 系統技術堆疊

1.1. 前端與後端整合框架 (Next.js 全端架構)

- 前端畫面與後端 API 均由 **Next.js 16** (搭配 **React 19**、**TypeScript**) App Router 統一承載。
- 伺服器端進入點 (Route Handlers, 位於 `app/api/` 目錄) 直接處理後端業務邏輯, 並直接與 PostgreSQL 資料庫對接, 無須依賴獨立的後端 API 伺服器。
- 正式部署採 Next.js standalone 輸出, 由 Node.js runtime 執行, 再由 IIS / Apache / Nginx 反向代理對外提供服務。

1.2. 前端樣式設計

- Vanilla CSS 系統 (不使用 TailwindCSS 框架, 採本地端 HSL 顏色變數與毛玻璃效果之 Premium 視覺設計, 以 `app/globals.css` 作為全域樣式基礎)。

1.3. 資料庫系統

- PostgreSQL。

1.4. 伺服器端資料庫與檔案處理

- **資料庫連接**: 使用 PostgreSQL 官方 `pg` 程式庫之連接池 (Pool) 進行資料庫操作 (透過 `lib/server/db.ts` 統一管理, 並以 `BEGIN / COMMIT / ROLLBACK` 實現交易機制)。
- **檔案與文件處理**: 伺服器端獨立處理實體檔案儲存、副檔名與 MIME Type 白名單檢核, 並使用 `pdf-lib` 進行 PDF 檔案處理與浮水印動態生成。

1.5. 資料表與欄位命名規則

本系統資料表與欄位採短表名與短欄位前綴。命名需符合既有 schema 文件風格, 例如 `dms_folders` 使用 `df_` 前綴、`dms_folder_acl` 使用 `dfa_` 前綴、`dms_folder_managers` 使用 `dfm_` 前綴。

文件管理與版本控管核心資料表命名如下：

資料表	用途	欄位前綴	主鍵欄位
<code>dms_doc</code>	文件主檔	<code>dd_</code>	<code>dd_id</code>
<code>dms_doc_ver</code>	文件版本	<code>ddv_</code>	<code>ddv_id</code>
<code>dms_file</code>	檔案後設資料	<code>dfi_</code>	<code>dfi_id</code>
<code>dms_ver_rev</code>	版本修訂對照表	<code>dvr_</code>	<code>dvr_id</code>

欄位命名原則如下：

- 主鍵採「資料表前綴 + id」格式，例如 `dms_doc.dd_id`、`dms_doc_ver.ddv_id`。
- 主鍵不得重複放入業務名詞。例如文件主檔主鍵使用 `dd_id`，不使用 `dd_doc_id`；文件版本主鍵使用 `ddv_id`，不使用 `ddv_ver_id`。
- 本專案資料庫不建立實體 **FOREIGN KEY** 約束。跨表關聯以欄位命名、索引、後端交易檢核與必要查詢條件維護，不依賴資料庫外鍵。
- 關聯欄位若只指向單一父表，沿用父表主鍵欄位名，例如 `dms_doc_ver.dd_id`、`dms_ver_rev.ddv_id`。
- 同一資料表若有多個關聯欄位指向同一父表，需加入用途語意，並保留被參照主鍵縮寫，例如 `ddv_pub_dfi_id` (正式發佈檔案) 與 `ddv_src_dfi_id` (原始編修檔案)。
- 欄位名稱不得包含句點。`dms_doc.dd_id` 僅代表 SQL 查詢中的「資料表名稱 + 欄位名稱」參照方式，實際欄位名稱為 `dd_id`。
- 文件版本不設 `ddv_status` 欄位。版本狀態需由 `ddv_eff_at`、`ddv_eff_to` 與 `ddv_cancel_at` 推導。
- 建立、更新、作廢欄位延續既有命名慣例，例如 `crtby`、`crtat`、`updby`、`updat`、`dc`、`dcby`、`dcat`。

常用縮寫規則如下：

業務語意	縮寫	範例
文件	<code>doc</code> 或 <code>dd</code> 前綴	<code>dms_doc</code> 、 <code>dd_id</code>
版本	<code>ver</code> 或 <code>ddv</code> 前綴	<code>dms_doc_ver</code> 、 <code>ddv_id</code>
檔案	<code>file</code> 或 <code>dfi</code> 前綴	<code>dms_file</code> 、 <code>dfi_id</code>
修訂	<code>rev</code>	<code>ddv_rev_date</code> 、 <code>dms_ver_rev</code>
生效	<code>eff</code>	<code>ddv_eff_at</code>
結束	<code>to</code>	<code>ddv_eff_to</code>
異動	<code>chg</code>	<code>ddv_chg_note</code>
廢止	<code>obs</code>	<code>dd_obs_at</code> 、 <code>dd_obs_reason</code>
撤回	<code>cancel</code>	<code>ddv_cancel_at</code> 、 <code>ddv_cancel_reason</code>
發佈	<code>pub</code>	<code>ddv_pub_dfi_id</code>
原始檔	<code>src</code>	<code>ddv_src_dfi_id</code>
作廢	<code>dc</code>	<code>dvr_dc</code>

文件管理與版本控管核心資料表欄位命名方向如下：

資料表	欄位方向
<code>dms_doc</code>	<code>dd_id</code> 、 <code>df_fid</code> 、 <code>dd_code</code> 、 <code>dd_title</code> 、 <code>dd_status</code> 、 <code>dd_obs_at</code> 、 <code>dd_obs_by</code> 、 <code>dd_obs_reason</code> 、 <code>dfi_id</code> 、 <code>dd_obs_src</code> 、 <code>dd_crtby</code> 、 <code>dd_crtat</code> 、 <code>dd_updby</code> 、 <code>dd_updat</code>
<code>dms_doc_ver</code>	<code>ddv_id</code> 、 <code>dd_id</code> 、 <code>ddv_seq</code> 、 <code>ddv_no</code> 、 <code>ddv_rev_date</code> 、 <code>ddv_eff_at</code> 、 <code>ddv_eff_to</code> 、 <code>ddv_chg_note</code> 、 <code>ddv_pub_dfi_id</code> 、 <code>ddv_src_dfi_id</code> 、

資料表	欄位方向
	ddv_cancel_at、ddv_cancel_by、ddv_cancel_reason、ddv_crtby、ddv_crtat、ddv_updbby、ddv_updat
dms_file	dfi_id、dfi_role、dfi_name、dfi_path、dfi_ext、dfi_mime、dfi_size、dfi_sha256、dfi_status、dfi_crtby、dfi_crtat
dms_ver_rev	dvr_id、ddv_id、dvr_base_ddv_id、dfi_id、dvr_note、dvr_dc、dvr_crtby、dvr_crtat、dvr_dcby、dvr_dcat
dms_audit_log	id、event_at、actor_uid、actor_name、actor_role、action、resource_type、resource_id、managed_folder_id、folder_id、document_id、version_id、result、ip_address、user_agent、request_id、reason、before_data、after_data、metadata

第 5 節稽核資料表採同一命名原則。全域稽核表使用 `dms_audit_log`；文件查閱、預覽與下載紀錄若獨立成表，使用 `dms_doc_access_log`。

2. 帳戶角色與權限矩陣 (基於資料夾的存取控制 Folder-based ACL)

系統採用「帳戶角色」與「資料夾管理身分」分離的權限模型。登入後的帳戶角色僅分為 `ADMIN` 與 `USER`；`FOLDER_MANAGER` 不作為登入角色，而是由 `dms_folder_managers` 依資料夾節點計算出的管理身分。原全站通用的「文件管理員 (MANAGER)」角色已全面廢除。

2.1. 系統管理員 (ADMIN)

- **職責：**全域系統管理、第一層資料夾建立、權限結構維護與例外處理。
- **根目錄管理權：**根目錄與第一層資料夾的建立權固定僅屬於系統管理員 (ADMIN)。根目錄文件建立權也固定僅屬於 ADMIN，系統需以 `folder_id = 0` 表示根目錄文件歸屬。根目錄文件查閱權固定為公開，任何已登入使用者皆可看見符合一般使用者文件生效條件的根目錄文件。
- **全站強制管理權：**ADMIN 對所有資料夾、ACL、文件與版本生命週期保有最高層級的管理與編輯權限。即使資料夾已指定資料夾管理員，或資料夾被設定為限閱，ADMIN 仍可執行建立子資料夾、修改資料夾屬性、封存資料夾、設定 ACL 與文件管理操作。
- **資料夾管理員指派規則：**ADMIN 可於任一資料夾節點指派一位或多位直接管理員。若清空某節點的直接管理員清單，表示該節點不新增獨立管理員，其有效管理員改由上層節點繼承而來。
- **存取限制豁免：**ADMIN 不受一般使用者 ACL 限制，且不得因錯誤 ACL 設定被鎖出系統管理範圍之外。

2.2. 資料夾管理員 (Folder Manager，計算型管理身分)

- **身分定義：**資料夾管理員不是帳戶角色。只要使用者被寫入某資料夾節點的 `dms_folder_managers`，即為該節點的直接管理員。
- **有效管理員計算：**某資料夾節點的有效管理員為「該節點本身與所有祖先節點直接管理員的聯集」。因此，上層資料夾管理員可管理其轄下所有子資料夾與文件。
- **空管理員清單規則：**空管理員清單只代表該節點不設定直接管理員，不代表取消上層繼承，也不代表改由一般使用者管理。
- **權限範圍：**
 - 在其有效管理範圍內，可建立子資料夾、修改資料夾名稱、設定資料夾直接管理員、封存資料夾、建立文件、上傳新版、版更、作廢文件與維護 ACL。

- 不得建立第一層資料夾；第一層資料夾建立固定由 ADMIN 執行。
- 不得於根目錄建立文件；根目錄文件建立固定由 ADMIN 執行。
- **ACL 授權管理**：資料夾管理員可將其有效管理範圍內的資料夾設定為公開或限閱。限閱時可授權特定部門或特定使用者。

2.3. 一般使用者 (USER)

- **職責**：文件查閱與下載。
- **預設權限**：採資安優先原則。一般使用者預設無法看見任何資料夾與一般資料夾文件。例外：位於根目錄 (`folder_id = 0`) 的文件固定公開，任何已登入使用者皆可看見符合文件生效條件的根目錄文件。
- **公開規則**：資料夾必須明確設定為公開，才可被一般使用者瀏覽。沒有 ACL 紀錄不代表公開。
- **限閱規則**：資料夾設定為限閱時，必須明確授權特定部門或特定使用者。若限閱資料夾沒有任何有效授權對象，則一般使用者不可見。
- **唯讀限制**：一般使用者僅具備唯讀、線上預覽 (PDF) 與下載權限，PDF 不提供下載，僅提供列印，只能下載「生效中」且「處於生效區間內」非 PDF 格式之最新版文件。一般使用者不得執行修改、建立、版更、作廢、封存與檢視歷史廢止紀錄等管理操作。
- **修訂對照表檢視規則**：任何具備文件查閱權限的使用者皆可檢視其可見文件版本之修訂對照表。

2.4. 資料夾 ACL 狀態

- `access_type = 1`：公開。一般使用者可瀏覽該資料夾，並可查閱符合文件生效條件的文件。
- `access_type = 2`：限閱。僅 ADMIN、有效資料夾管理員、被授權部門成員與被授權使用者可瀏覽。
- 新建資料夾預設為 `access_type = 2`。若未設定授權部門或授權使用者，則只有 ADMIN 與有效資料夾管理員可見。
- 根目錄不是實體資料夾節點，不套用 `dms_folder_acl`。位於根目錄的文件固定公開查閱，但建立、版更、廢止與維護仍固定僅限 ADMIN。

3. 資料夾結構管理

3.1. 基本狀態

資料夾狀態 (`status`) 以代碼儲存，分為三種：

- `1`：正常（使用中）。
- `2`：封存（已封存）。
- `0`：已刪除（作廢）。

3.2. 刪除（作廢）條件

若資料夾底下沒有任何有效物件（包含正常狀態子資料夾與有效文件），系統開放管理員執行刪除（作廢）操作，並將狀態代碼更新為 `0`。已封存、已作廢或已廢止的歷史紀錄不阻擋空資料夾刪除（作廢）。

空資料夾的管理操作應直接提供「刪除（作廢）」功能，不需顯示「封存」項目。前端操作清單需將「刪除（作廢）」與「封存」視為互斥入口：資料夾底下沒有任何有效物件時，只顯示「刪除（作廢）」；資料夾底下仍有任何正常狀態子資料夾或有效文件時，才依封存規則顯示「封存」入口。

3.3. 封存資料夾（Recursive Archive，遞迴封存）

- 當管理員對某一正常狀態 (1) 的資料夾進行封存時，系統將執行遞迴封存，一併將其下所有子資料夾之狀態更新為 2。
- **文件聯動廢止**：被封存資料夾及其子資料夾下的所有文件，其文件主檔狀態將被自動更新為 **Obsolete** (已廢止)，並於文件主檔註記廢止時間與廢止原因。文件版本的有效期間與撤回紀錄不因資料夾封存而改寫，以確保版本歷史維持原始紀錄。
- **文件庫顯示規則**：已封存資料夾不得顯示於文件庫的資料夾樹、資料夾清單與一般文件瀏覽流程中。已封存資料夾與其文件仍保留於資料庫及檔案儲存中，未來由「資源回收區」或其他專用功能區檢視。
- **不可逆性**：封存操作為不可逆，檔案實體與歷史紀錄將永久保留於伺服器中存檔備查。

4. 文件管理與版本控管

DMS 不負責文件簽核流程。文件內容之擬稿、會簽、核准均由 BPM 系統處理；DMS 僅負責保存 BPM 簽核完成後，由資料夾管理員上傳之正式文件，並管理其版本、預約生效、有效期間、撤回、廢止、查閱與下載規則。

本系統採「預約生效模式」與「版本有效期間模型」。資料夾管理員可上傳已完成 BPM 核准之正式文件，並指定今天或未來日期作為版本生效日。一般使用者只會看到目前有效版本，不會看到尚未生效、已結束、已撤回或已廢止文件的內容。

4.1. 文件主檔與版本分離

系統採「文件主檔」與「文件版本」分離設計。

- **文件主檔**：代表一份文件本身，例如文件編號、文件名稱、所屬資料夾、文件是否廢止、廢止原因與廢止公文等資料。
- **文件版本**：代表該文件某一次正式發佈的版本，包含版本號、修訂日期、生效日期、結束日期、異動說明、正式發佈檔案、撤回紀錄與修訂對照表等資料。

同一份文件可以有多个版本，但同一時間對一般使用者僅能解析出一個目前有效版本。

版本不另設版本狀態欄位。版本是否尚未生效、目前有效、已成為歷史版本或已撤回，均由版本有效期間與撤回欄位推導。

4.2. 文件主檔狀態

文件主檔狀態只表示整份文件是否仍具備流通資格，不表示哪一個版本目前有效。

- **Effective** (有效的)：文件未廢止。若存在符合有效期間的版本，該版本可依權限供一般使用者查閱或下載。
- **Obsolete** (已廢止)：文件已廢止，不再流通。

文件被手動廢止或因資料夾封存而廢止時，應在文件主檔註記，不改寫既有版本的有效期間。

4.3. 版本有效期間規則

每一個文件版本以有效期間判斷是否為目前有效版本。

版本有效期間欄位如下：

- **生效時間 effective_at (生效時間)**：版本開始生效時間，必填。
- **結束時間 effective_until (結束時間)**：版本結束時間，可空白。空白代表該版本尚未被後續版本排定取代。

一般使用者查詢目前版本時，版本必須同時符合以下條件：

- 文件主檔狀態為 **Effective** (有效的)。
- 版本未被撤回。
- **effective_at** (生效時間) 欄位值小於或等於目前時間。
- **effective_until** (結束時間) 為空白，或目前時間小於 **effective_until** (結束時間)。
- 位置權限符合下列任一條件：
 - 位於根目錄 (**folder_id** = 0)。
 - 所屬一般資料夾為正常狀態，且使用者符合該管理資料夾節點 ACL 授權規則。

版本有效期間採半開區間：

```
effective_at (生效時間), effective_until (結束時間)
effective_at <= 目前時間
AND (effective_until IS NULL OR 目前時間 < effective_until)
```

同一份文件同一時間不得存在兩筆對一般使用者同時有效的版本。

文件仍處於第一版狀態時，資料夾管理員可執行「刪除文件」。刪除文件僅允許在文件主檔仍有效、版本總數為 1、未撤回版本總數為 1，且該版本 **ddv_seq** = 1 時執行；刪除後仍需保留全域稽核紀錄。

若版本 **effective_at** (生效時間) 晚於目前時間，該版本屬於尚未生效版本，僅資料夾管理員可見。系統不得因尚未生效版本存在，而讓一般使用者看不到上一個仍在有效期間內的版本。

4.4. 版本欄位規則

每一個文件版本需保存以下資訊：

- **版本號**：可空白，由管理員依實際文件規範自行填寫。
- **修訂日期**：必填，代表該文件內容正式修訂日期。
- **生效日期** **effective_at (生效時間)**：必填，可指定今天或未來日期，不允許早於上傳日期。
- **結束日期** **effective_until (結束時間)**：由系統於預約新版或立即版更時寫入，不提供使用者手動填寫。
- **異動說明**：必填，欄位需支援下拉片語選擇與手動輸入。使用者可從下拉選單選取常用異動說明項目，也可直接輸入自訂說明內容。下拉選單至少包含：
 - 初版發行
 - 內容修訂
 - 格式調整
 - 錯字或文字修正
 - 法規或制度更新
 - 流程變更
 - 組織或職責調整
 - 表單欄位調整
 - 附件或範本更新
 - 週期性檢討，內容無異動
 - 文件整併
 - 文件拆分
 - 替代舊版文件

- **正式發佈檔案**：必填，代表該版本對外流通的正式檔案。
- **撤回日期 `cancelled_at` (撤回日期)**：資料夾管理員執行「撤回版本並回復前版」時自動寫入。
- **撤回人員 `cancelled_by` (撤回人員)**：資料夾管理員執行「撤回版本並回復前版」時自動寫入。
- **撤回原因 `cancel_reason` (撤回原因)**：資料夾管理員執行「撤回版本並回復前版」時必填。

日期輸入欄位通用規則如下：

- 凡需由使用者輸入日期的欄位，均需提供月曆介面選擇。
- 月曆介面不得取代後端日期檢核；後端仍需依欄位規則驗證日期是否允許。

生效日期之時間粒度規則如下：

- 若生效日期為上傳當日，版本於上傳交易完成後立即生效。
- 若生效日期晚於上傳當日，版本於該日期 `00:00:00` 生效。
- 系統以伺服器時間作為版本有效期間判斷基準，前端不得自行判斷版本是否生效。
- 本系統僅在本地環境使用，所有「目前時間」均指後端伺服器本地時間，資料庫保存系統本地時間即可。

文件主檔需保存以下廢止資訊：

- **廢止日期**：手動廢止或資料夾封存造成文件廢止時自動寫入。
- **廢止原因**：手動廢止或系統自動廢止時寫入。
- **廢止公文或核准文件**：手動廢止時必填；資料夾封存造成的系統自動廢止不強制要求。

文件類型暫不納入規格，不依規章、SOP、表單或 ISO 文件類型套用不同規則。

4.5. 正式發佈檔案格式規則

每一個文件版本必須上傳一個「正式發佈檔案」。

系統採白名單方式限制可上傳格式，預設允許 PDF、Microsoft Office 常用文件格式、常用圖檔格式與常用 HTML 文件格式。

允許之正式發佈檔案格式：

- PDF：`.pdf`
- Word：`.docx`、`.doc`
- Excel：`.xlsx`、`.xls`
- PowerPoint：`.pptx`、`.ppt`
- 圖檔：`.jpg`、`.jpeg`、`.png`、`.gif`、`.tif`、`.tiff`、`.webp`
- HTML：`.html`、`.htm`、`.mhtml`

系統不提供 Word、Excel、PowerPoint、HTML 與 PDF 之間的自動轉檔功能，也不自動變更檔案格式。所有正式發佈檔案皆以資料夾管理員上傳之檔案為準。

為降低資安風險，系統不允許上傳含巨集、可執行、腳本、安裝檔、捷徑或壓縮封裝類型檔案。

禁止之高風險檔案類型包含但不限於：

- Office 巨集格式：`.docm`、`.dotm`、`.xlsm`、`.xltm`、`.xlam`、`.pptm`、`.potm`、`.ppam`
- 可執行與系統檔：`.exe`、`.msi`、`.dll`、`.com`、`.scr`
- 腳本與指令檔：`.bat`、`.cmd`、`.ps1`、`.vbs`、`.js`、`.hta`
- 捷徑與設定檔：`.lnk`、`.reg`

- 壓縮檔與封裝檔：`.zip`、`.rar`、`.7z`、`.tar`、`.gz`

檔案上傳時，系統應檢查副檔名與 MIME Type (媒體類型)；若副檔名或 MIME Type (媒體類型) 不符合白名單，應拒絕上傳。

4.6. PDF 文件規則

若正式發佈檔案為 PDF (可攜式文件格式)：

- 一般使用者僅能線上預覽 PDF。
- 一般使用者不提供 PDF 下載按鈕。
- 一般使用者即使直接呼叫 PDF 正式原檔下載 API (應用程式介面)，或以 URL (網址) 猜測檔案路徑，後端仍必須依角色與 ACL (存取控制清單) 拒絕下載。
- PDF 預覽畫面必須顯示浮水印。
- 浮水印必須由 Next.js Route Handler 在伺服器端讀取 PDF 原始檔後動態套用，不得由瀏覽器端產生或合成，避免客戶端繞過浮水印取得預覽內容。
- 浮水印使用內建 `Noto Sans TC` 字型資源產生中文字形，並以半透明、斜向方式置中顯示於每一頁；輸出結果僅供當次預覽，不得覆寫磁碟上的正式 PDF 原檔。
- 使用者如需紙本或另存 PDF，可使用瀏覽器列印功能。
- 資料夾管理員可下載正式 PDF 原檔。
- 資料夾管理員介面需顯示「上傳原始檔案」按鈕。
- 原始檔案為選填，供日後編修、追溯使用。
- 原始檔案格式必須符合 4.5「正式發佈檔案格式規則」之白名單規範。
- 原始檔案僅限資料夾管理員可見與下載，一般使用者不可見。

浮水印內容應包含：

- 使用者姓名及帳號
- 預覽時間，固定使用 `Asia/Taipei` 時區
- 用戶端 IP；伺服器依序讀取 `X-Forwarded-For`、`X-Real-IP`、`CF-Connecting-IP`、`True-Client-IP` 與標準 `Forwarded` header，均無有效值時顯示「無法判定」
- 文件編號
- 「內部文件，禁止外流」

4.7. 非 PDF 文件規則

若正式發佈檔案為 HTML 文件格式 (`.html`、`.htm`、`.mhtml`)：

- 系統提供開新視窗預覽。
- 系統不套用浮水印。

若正式發佈檔案不是 PDF (可攜式文件格式)，也不是 HTML 文件格式，例如 Word (文字文件)、Excel (試算表)、PowerPoint (簡報) 或圖檔：

- 系統不提供線上預覽。
- 系統不套用浮水印。
- 一般使用者僅提供下載。
- 不顯示「上傳原始檔案」按鈕。
- 該檔案即為此版本之正式發佈檔案。

此規則主要適用於表單、範本、試算表、簡報或需要使用者下載後編輯、填寫、使用之文件。

4.8. 新建文件

文件可建立於一般資料夾或根目錄，但權限來源不同：

- **一般資料夾建立文件**：有效資料夾管理員與 ADMIN 可在其管理範圍內建立文件。
- **根目錄建立文件**：僅 ADMIN 可在根目錄建立文件。前端僅對 ADMIN 顯示根目錄「新建文件」入口；後端 API 必須再次驗證登入角色，不得只依賴前端防呆。
- **根目錄文件公開規則**：位於根目錄的文件固定公開查閱。任何已登入使用者進入文件庫根層時，皆可看見符合文件主檔狀態、版本生效期間與 PDF / HTML / 非 PDF 操作規則的根目錄文件。
- **根目錄識別規則**：建立或查詢根目錄文件時，系統以 `folder_id = 0` 表示根目錄。一般資料夾文件則使用實際 `dms_folders.df_fid`。
- **拒絕規則**：非 ADMIN 若直接呼叫 API 嘗試以 `folder_id = 0` 建立文件，後端必須拒絕並回傳權限不足訊息，同時寫入權限不足稽核紀錄。

管理員建立文件時：

1. 建立文件主檔。
2. 上傳第一個正式發佈檔案。
3. 填寫修訂日期、生效日期，並選取或輸入異動說明。
 - 生效日期預設為當下日期。
4. 版本號可填寫或留空。
5. 文件主檔狀態設為 `Effective` (有效的)。
6. 第一個文件版本寫入 `effective_at` (生效時間)。
7. 第一個文件版本的 `effective_until` (結束時間) 保持空白。
8. 若第一個版本尚未到生效日期，一般使用者不可看到該文件；資料夾管理員可看到此尚未生效版本。

4.9. 文件版更與預約生效

資料夾管理員上傳新版文件時：

1. 新增一筆文件版本。
2. 上傳新的正式發佈檔案。
3. 填寫修訂日期、生效日期，並選取或輸入異動說明。
4. 版本號可填寫或留空。
5. 新版本寫入 `effective_at` (生效時間)。
6. 新版本的 `effective_until` (結束時間) 保持空白。
7. 系統將上一個未撤回版本的 `effective_until` (結束時間) 設為新版本的 `effective_at` (生效時間)。

若新版本生效日期晚於目前時間：

- 一般使用者在新版本生效前仍看到上一個版本。
- 新版本到達生效時間後，一般使用者看到新版本。
- 資料夾管理員在新版本生效前即可查閱與下載該版本。

若新版本生效日期為上傳當日：

- 上一個版本的 `effective_until` (結束時間) 設為新版本生效時間。
- 一般使用者於版更完成後看到新版本。

同一份文件同一時間只允許一個尚未撤回且尚未生效的預約版本。此限制必須由後端 API (應用程式介面) 檢查，不得僅依賴前端畫面防呆；判斷條件為同一 `document_id` (文件識別碼) 下不存在 `cancelled_at` (撤回日期)

為空白且 **effective_at** (生效時間) 晚於目前時間的版本。若已存在預約版本，資料夾管理員必須先撤回該版本並回復前版，才能再上傳新的預約版本。

版本生效不需另行改寫版本狀態；系統以 **effective_at** (生效時間)、**effective_until** (結束時間) 與 **cancelled_at** (撤回日期) 判斷目前有效版本。

4.10. 撤回版本並回復前版

資料夾管理員可對最新一筆未撤回版本執行「撤回版本並回復前版」。

不論該版本是否已到生效日期，撤回時均依下列規則處理：

- 被撤回版本寫入 **cancelled_at** (撤回日期)、**cancelled_by** (撤回人員) 與 **cancel_reason** (撤回原因)。
- 被撤回版本的正式發佈檔案、異動說明與修訂對照表永久保留。
- 前一個版本的 **effective_until** (結束時間) 清空。
- 撤回完成後，一般使用者只看到前一個版本。
- 被撤回版本僅資料夾管理員可查閱，作為稽核紀錄。

若被撤回版本曾經生效，系統不得刪除或改寫其 **effective_at** (生效時間)。該版本的 **effective_at** (生效時間) 與 **cancelled_at** (撤回日期) 共同表示該版本曾經流通，並於撤回時間停止對一般使用者可見。前一版 **effective_until** (結束時間) 清空僅用於目前有效版本解析，不代表抹除新版曾經流通之事實，實際時間線需由全域稽核紀錄保存。

已撤回版本不得重新啟用。若未來要再次使用相同內容，必須走一般文件版更流程重新建立版本。

4.11. 手動廢止文件

資料夾管理員可手動廢止整份文件。

手動廢止時：

- 必須填寫廢止原因。
- 必須上傳廢止公文或核准文件。
- 文件主檔狀態改為 **Obsolete** (已廢止)。
- 文件主檔寫入廢止日期。
- 文件主檔寫入廢止原因。
- 文件主檔保存廢止公文或核准文件。
- 既有版本的 **effective_at** (生效時間)、**effective_until** (結束時間) 與撤回紀錄不改寫。

廢止成功後，一般使用者不再看到該文件。

文件主檔狀態為 **Obsolete** (已廢止) 後，不得再執行上傳新版、預約生效、撤回版本並回復前版等版本生命週期操作。若需重新啟用相同內容，應建立新文件或另行定義重新發行流程。

4.12. 資料夾封存造成的文件廢止

當資料夾被封存時，其下文件需同步處理：

- 文件主檔狀態改為 **Obsolete** (已廢止)。
- 文件主檔寫入廢止日期。
- 文件主檔自動填入廢止原因：「因所屬資料夾封存而自動廢止」。

- 此類系統自動廢止不強制要求上傳廢止公文。
- 既有版本的 `effective_at` (生效時間)、`effective_until` (結束時間) 與撤回紀錄不改寫。

資料夾封存與文件主檔廢止註記需於同一個資料庫交易內完成。若任一文件廢止註記失敗，整個封存交易必須回復。

因資料夾封存而變為 `Obsolete` (已廢止) 的文件，同樣不得再執行上傳新版、預約生效、撤回版本並回復前版等版本生命週期操作。

4.13. 一般使用者權限

一般使用者僅能看到符合下列文件狀態與版本條件的文件：

- 文件主檔狀態為 `Effective` (有效的) 的文件。
- 未被撤回且目前時間落在有效期間內的目前版本。

一般使用者可見文件的位置權限需符合下列任一條件：

- 位於根目錄 (`folder_id = 0`)。
- 所屬一般資料夾為正常狀態，且符合該管理資料夾節點 ACL 授權規則。

一般使用者不可看到：

- 尚未生效版本。
- 歷史版本。
- 已撤回版本。
- 已廢止文件。
- 廢止原因。
- 廢止公文。
- PDF 原始檔。
- PDF 文件之原始編修檔案。

一般使用者對 PDF、HTML 與其他非 PDF 文件之操作如下：

- PDF (可攜式文件格式)：僅可線上預覽，不提供下載按鈕；後端 API (應用程式介面) 亦不得提供一般使用者下載正式 PDF 原檔。
- HTML 文件格式 (`.html`、`.htm`、`.mhtml`)：提供開新視窗預覽。
- 其他非 PDF 文件 (不是可攜式文件格式，也不是 HTML 文件格式)：僅提供下載，不提供線上預覽。

一般使用者的資料夾瀏覽與搜尋結果均須套用上述規則；不得在搜尋結果中揭露無權限、尚未生效、歷史、已撤回或已廢止文件的標題、版本、檔名或其他可識別資訊。

4.14. 資料夾管理員權限

ADMIN 除具備所有資料夾管理員權限外，另可於根目錄建立文件。根目錄文件不屬於任何資料夾管理員繼承範圍，非 ADMIN 不得建立、版更、廢止或維護根目錄文件；非 ADMIN 僅可依一般使用者權限查閱符合生效條件的根目錄文件。

資料夾管理員可：

- 新增子資料夾。
- 建立文件。

- 上傳新版。
- 上傳預約生效版本。
- 撤回最新版本並回復前版。
- 查看所有版本歷史。
- 查看尚未生效版本。
- 查看已撤回版本。
- 下載歷史版本檔案。
- 下載 PDF 正式原檔。
- 上傳與下載 PDF 文件之原始檔案。
- 查看廢止原因。
- 下載廢止公文。
- 手動廢止文件。

4.15. 修訂對照表

系統應支援修訂對照表功能。修訂對照表亦可稱為「修訂前後對照表」。

依文件簽核流程原則，文件更版時應於簽核資料中附上修訂前後對照表。DMS 不負責產生修訂前後對照表，僅提供資料夾管理員於上傳新版時選擇是否一併保存修訂前後對照表。

DMS 不強制要求每次上傳新版時必須提供修訂前後對照表。未上傳修訂前後對照表不得阻擋文件建檔、上傳新版或預約生效；系統可提示資料夾管理員確認是否需要保存修訂前後對照表。資料夾管理員可於版本建立後後補或更新該版本的修訂前後對照表。

修訂對照表隸屬於單一文件版本。每一份修訂對照表需記錄本次版本與比較基準版本，預設以被本次版本排定取代的前一個版本作為比較基準。

修訂明細可包含：

- 修訂章節或頁碼。
- 修訂前內容。
- 修訂後內容。
- 修訂原因。
- 備註。

修訂對照表屬於 DMS 的版本紀錄，不取代正式發佈檔案內容。

任何具備文件查閱權限的使用者皆可檢視其可見文件版本之修訂對照表。若版本被撤回，該版本之修訂對照表仍需保留，並依版本可見性與歷史紀錄權限控管；無權檢視該版本者，不得透過修訂對照表取得已撤回版本內容。

5. 全域稽核紀錄規格

DMS 內所有會影響資料、權限、安全、文件流通或使用者存取軌跡的行為，都應寫入全域稽核紀錄。稽核紀錄不僅涵蓋文件版本生命週期，也涵蓋登入、查閱、下載、資料夾屬性、資料夾管理員、ACL (存取控制清單) 與權限不足嘗試等事件。

5.1. 稽核設計原則

- 稽核紀錄採追加寫入模式，系統一般功能不提供修改或刪除稽核紀錄的操作。
- 稽核紀錄應由後端統一寫入，不得只依賴前端事件紀錄。

- 管理類異動成功時，主要資料異動與稽核紀錄應在同一個短交易中完成；若稽核紀錄寫入失敗，該管理異動應回復。
- 查閱、預覽與下載屬高頻事件，仍須留下紀錄；實作上可寫入同一張 `dms_audit_log` (稽核紀錄表)，或獨立為 `dms_doc_access_log` (文件存取紀錄表) 與分區表，但查詢介面上應視為同一套全域稽核資料。
- 稽核紀錄應保存足以還原「誰、於何時、從何處、對何資料、做了什麼、結果如何」的資訊。

5.2. 必須記錄的事件範圍

下列事件均需寫入全域稽核紀錄：

- 驗證事件：`AUTH_LOGIN_SUCCESS` (登入成功)、`AUTH_LOGIN_FAILED` (登入失敗)、`AUTH_LOGOUT` (登出)。
- 存取事件：`FOLDER_VIEWED` (資料夾瀏覽)、`DOCUMENT_PREVIEWED` (文件預覽)、`DOCUMENT_DOWNLOADED` (文件下載)、`DOCUMENT_DOWNLOAD_DENIED` (文件下載被拒絕)。
- 資料夾事件：`FOLDER_CREATED` (建立資料夾)、`FOLDER_UPDATED` (修改資料夾)、`FOLDER_ARCHIVED` (封存資料夾)、`FOLDER_DELETED` (作廢資料夾)。
- 權限事件：`FOLDER_MANAGER_UPDATED` (資料夾管理員異動)、`FOLDER_ACL_UPDATED` (資料夾存取控制清單異動)。
- 文件事件：`DOCUMENT_CREATED` (建立文件)、`DOCUMENT_VERSION_CREATED` (建立文件版本)、`DOCUMENT_VERSION_SCHEDULED` (預約生效版本)、`DOCUMENT_VERSION_CANCELLED` (撤回文件版本)、`DOCUMENT_PREVIOUS_VERSION_RESTORED` (回復前一版)、`DOCUMENT_DELETED` (刪除文件)、`DOCUMENT_OBSOLETE` (手動廢止文件)、`DOCUMENT_AUTO_OBSOLETE_BY_FOLDER_ARCHIVE` (因資料夾封存自動廢止文件)。
- 失敗與拒絕事件：`PERMISSION_DENIED` (權限不足)、`OPERATION_FAILED` (操作失敗)。

5.3. 稽核資料欄位

全域稽核紀錄建議至少保存下列欄位：

欄位	說明
<code>id</code> (流水號)	稽核紀錄唯一識別碼。
<code>event_at</code> (事件時間)	事件發生時間，以後端伺服器本地時間寫入。
<code>actor_uid</code> (操作者帳號)	執行動作的使用者帳號；未登入或登入失敗時可依實際情境留空或記錄輸入帳號。
<code>actor_name</code> (操作者姓名)	執行動作的使用者姓名。
<code>actor_role</code> (操作者角色)	執行動作當下的登入角色，僅記錄 <code>ADMIN</code> (系統管理員) 或 <code>USER</code> (一般使用者)。資料夾管理員屬於計算型管理身分，不寫入為登入角色。
<code>action</code> (動作代碼)	本次事件類型，例如 <code>DOCUMENT_VERSION_CANCELLED</code> (撤回文件版本)。
<code>resource_type</code> (資源類型)	被操作的資源類型，例如 <code>AUTH</code> (驗證)、 <code>FOLDER</code> (資料夾)、 <code>DOCUMENT</code> (文件主檔)、 <code>VERSION</code> (文件版本)、 <code>ACL</code> (存取控制清單)。
<code>resource_id</code> (資源識別碼)	被操作資源的主要識別碼。

欄位	說明
<code>managed_folder_id</code> (管理資料夾節點識別碼)	事件所屬管理資料夾節點識別碼，便於依權限範圍查詢。
<code>folder_id</code> (資料夾識別碼)	事件所屬資料夾識別碼。
<code>document_id</code> (文件主檔識別碼)	事件所屬文件主檔識別碼。
<code>version_id</code> (文件版本識別碼)	事件所屬文件版本識別碼。
<code>result</code> (執行結果)	<code>SUCCESS</code> (成功)、 <code>FAILED</code> (失敗) 或 <code>DENIED</code> (拒絕)。
<code>ip_address</code> (來源位址)	使用者來源 IP 位址。
<code>user_agent</code> (瀏覽器識別資訊)	瀏覽器或用戶端識別資訊。
<code>request_id</code> (請求追蹤識別碼)	後端請求追蹤用識別碼，便於除錯與串接日誌。
<code>reason</code> (原因)	使用者填寫或系統產生的原因，例如撤回原因、廢止原因或拒絕原因。
<code>before_data</code> (變更前資料)	異動前快照，可使用 <code>JSONB</code> (JSON 二進位格式) 保存。
<code>after_data</code> (變更後資料)	異動後快照，可使用 <code>JSONB</code> (JSON 二進位格式) 保存。
<code>metadata</code> (額外資料)	其他補充資訊，例如檔名、MIME Type (媒體類型)、版本號、下載類型、浮水印內容或錯誤訊息。

5.4. 查閱、預覽與下載紀錄

一般使用者與資料夾管理員的查閱、預覽、下載行為均需記錄。

- PDF (可攜式文件格式) 預覽應記錄 `DOCUMENT_PREVIEWED` (文件預覽)，並於 `metadata` (額外資料) 中保存文件編號、版本、預覽時間與浮水印相關資訊。
- HTML 文件格式 (`.html`、`.htm`、`.mhtml`) 開新視窗預覽應記錄 `DOCUMENT_PREVIEWED` (文件預覽)，並保存檔名、副檔名、MIME Type (媒體類型)、文件版本與預覽結果。
- 非 PDF 文件下載應記錄 `DOCUMENT_DOWNLOADED` (文件下載)，並保存檔名、副檔名、MIME Type (媒體類型)、文件版本與下載結果。
- 資料夾管理員下載 PDF 正式原檔、PDF 原始編修檔、歷史版本檔案或廢止公文時，均需記錄 `DOCUMENT_DOWNLOADED` (文件下載)，並於 `metadata` (額外資料) 中標示 `file_purpose` (檔案用途)，例如 `PUBLISHED_FILE` (正式發佈檔案)、`SOURCE_FILE` (原始編修檔)、`OBSOLETE_APPROVAL_FILE` (廢止公文)。
- 一般使用者嘗試直接呼叫 PDF 正式原檔下載 API (應用程式介面) 時，後端應拒絕並記錄 `DOCUMENT_DOWNLOAD_DENIED` (文件下載被拒絕) 或 `PERMISSION_DENIED` (權限不足)。

5.5. 權限異動與資料夾異動紀錄

資料夾屬性、資料夾管理員與 ACL (存取控制清單) 異動需保留變更前後資料。

- 建立、修改、作廢或封存資料夾時，應記錄資料夾名稱、父層資料夾、狀態、異動人員與異動結果。
- 修改管理資料夾節點的資料夾管理員時，應於 `before_data` (變更前資料) 與 `after_data` (變更後資料) 保存管理員清單差異。

- 修改管理資料夾節點 ACL (存取控制清單) 時，應保存授權模式，例如公開、特定群組或特定使用者，以及群組與使用者清單差異。
- 若因權限不足導致資料夾或 ACL (存取控制清單) 修改被拒絕，仍需記錄拒絕事件。

5.6. 文件與版本生命週期紀錄

文件主檔與文件版本的生命週期異動，均需寫入全域稽核紀錄。

- 新建文件與上傳第一版時，需記錄文件主檔、第一筆版本、正式發佈檔案與生效時間。
- 上傳新版或預約生效版本時，需記錄新版本資料，以及前一版本 `effective_until` (結束時間) 被設定為新版本 `effective_at` (生效時間) 的異動。
- 撤回版本並回復前版時，需記錄 `DOCUMENT_VERSION_CANCELLED` (撤回文件版本) 與 `DOCUMENT_PREVIOUS_VERSION_RESTORED` (回復前一版)，並在 `before_data` (變更前資料) 與 `after_data` (變更後資料) 保存被撤回版本與前一版 `effective_until` (結束時間) 的變化。
- 刪除第一版文件時，需記錄 `DOCUMENT_DELETED` (刪除文件)，並保存操作者、所屬資料夾、文件主檔識別碼、第一版版本識別碼、刪除原因與請求來源資訊。
- 手動廢止文件時，需記錄廢止原因、廢止公文、文件主檔狀態從 `Effective` (有效的) 改為 `Obsolete` (已廢止) 的異動。
- 因資料夾封存自動廢止文件時，需記錄 `DOCUMENT_AUTO_OBSOLETE_BY_FOLDER_ARCHIVE` (因資料夾封存自動廢止文件)，並保存來源資料夾與系統自動廢止原因。

5.7. 失敗與拒絕存取紀錄

系統不只記錄成功事件，也需記錄失敗或被拒絕的嘗試。

- 登入失敗需記錄 `AUTH_LOGIN_FAILED` (登入失敗)，但不得記錄明文密碼。
- 權限不足、ACL (存取控制清單) 不符、角色不符、文件狀態不允許、版本狀態不允許等情況，均需記錄 `PERMISSION_DENIED` (權限不足) 或對應的拒絕事件。
- 後端驗證失敗、資料版本衝突、檔案格式不符或資料庫異常等情況，應記錄 `OPERATION_FAILED` (操作失敗)，並於 `metadata` (額外資料) 保存可供維護人員判讀的錯誤摘要。

5.8. 稽核紀錄保存與不可竄改原則

- 稽核紀錄未另訂保存期限前，原則上永久保存。
- 系統前端不提供稽核紀錄修改或刪除功能。
- 後端應限制一般 API (應用程式介面) 不可更新或刪除稽核紀錄，僅允許查詢。
- 高權限維護人員若需進行資料庫層級維護，應另以資料庫備份、維護紀錄或外部程序留存軌跡，不得透過 DMS 一般功能竄改稽核紀錄。

6. API (應用程式介面) 對接與部署設定規格

V6 採用 Next.js 全端架構。瀏覽器端只呼叫同源 Next.js `/api/*` 路由；Route Handler 於伺服器端直接執行 PostgreSQL 存取、權限判斷、檔案處理與稽核紀錄。瀏覽器不得直接連線資料庫，也不得持有資料庫帳密、session 密鑰或實體檔案路徑。

6.1. 目前正式架構

```

使用者瀏覽器
  ↓
IIS / Apache / Nginx
  ↓
Next.js standalone server
  ↳ PostgreSQL
  ↳ 本機或指定磁碟上的文件儲存目錄

```

- **瀏覽器責任**：渲染操作畫面、呼叫同源 `/api/*` 路由、接收下載或預覽檔案 stream。
- **Next.js 責任**：提供 App Router 畫面、Route Handler API、`HttpOnly Cookie` session 管理，並於伺服器端執行 PostgreSQL 存取、權限判斷、文件版本生命週期、PDF / 非 PDF 規則、稽核紀錄、檔案存取控制，以及檔案 stream 與 response header 輸出。
- **Web Server 責任**：IIS / Apache / Nginx 僅作為正式環境對外入口，將請求反向代理到 Next.js server。

6.2. 正式環境設定檔

V6 不再使用獨立後端主機網址。正式環境設定改由 Next.js server 讀取 `.env.production`（或環境變數）。

```

DATABASE_URL=postgres://user:password@host:port/database
PGPOOL_MAX=10
DMS_STORAGE_ROOT=.storage
DMS_LEGACY_STORAGE_ROOT=
SESSION_COOKIE_NAME=dms_session
SESSION_SECRET=請改成至少 32 字元的隨機字串
SESSION_MAX_AGE_SECONDS=28800
SESSION_COOKIE_SECURE=true
DMS_NEXT_HOST=127.0.0.1
DMS_NEXT_PORT=3000

```

- **DATABASE_URL** PostgreSQL 資料庫連線字串（包含帳密、主機、埠與資料庫名稱）。
- 未設定 **DATABASE_URL** 時，可改用 **PGHOST**、**PGPORT**、**PGDATABASE**、**PGUSER**、**PGPASSWORD** 分別設定 PostgreSQL 連線資訊。
- **PGPOOL_MAX** 連線池最大連線數，預設為 10。
- **DMS_STORAGE_ROOT** 為 V6 新上傳檔案的根目錄。完整路徑直接使用；相對路徑以 Next.js 執行目錄 `process.cwd()` 為基準解析；未設定時預設為 `./storage`。
- **DMS_LEGACY_STORAGE_ROOT** 為既有 DMS 檔案的相容讀取根目錄，可使用完整路徑或相對路徑；不需讀取舊檔案時保持空白。
- 新上傳檔案依 **YYYYMM** 月份建立子目錄，磁碟檔名使用時間戳記與 UUID 組合；資料庫保存相對儲存路徑，不向瀏覽器揭露實體絕對路徑。
- **SESSION_SECRET** 必須使用正式環境專用隨機字串。
- **SESSION_COOKIE_SECURE=true** 時，Web Server 對外入口必須使用 HTTPS。
- **DMS_NEXT_HOST** 建議固定為 `127.0.0.1`，避免 Next.js server 直接暴露到使用者網段。
- **DMS_NEXT_PORT** 必須與 Web Server 反向代理目標一致。

6.3. 登入連線規格與失敗處理

- **瀏覽器登入請求**：

- **Request URL** : `/api/login`
- **Method** : `POST`
- **Payload (Body)** : `{"uid": "使用者帳號", "pwd": "使用者密碼"}`
- **Content-Type** : `application/json`
- **Next.js 伺服器端行為** :
 - Route Handler 解析請求中的帳號密碼。
 - Route Handler 直接查詢 PostgreSQL 資料庫中的使用者與系統管理員資料 (透過 `lib/server/auth.ts`)。
 - 驗證通過後，將使用者資訊與管理員身分封裝成 session payload。
 - session payload 以 `HttpOnly Cookie` 加密保存，瀏覽器端 JavaScript 不可讀取敏感資料。
- **登入頁面欄位** :
 - 登入畫面使用共用 `Modal` 元件顯示，禁止點擊背景遮罩關閉，且不顯示右上角關閉按鈕。
 - 進入登入畫面時，帳號與密碼輸入欄位預設保持空白。
 - 網頁初次載入、或點選登出時，滑鼠游標會自動定位在「使用者帳號」輸入框。
- **驗證失敗處理** :
 - 當 API 回傳 `success: false` 時，系統會於介面顯示後端返回的 `error` 訊息。
 - 自動將「使用者帳號」與「密碼」輸入框之內容清空，且游標焦點會重新自動定位於「使用者帳號」輸入框。

6.4. Token (權杖)、Session 與路由管理

- **Session 保存方式** :
 - 使用者登入資訊不存放於 `sessionStorage`，防止 XSS 竊取。
 - 登入驗證資訊不放在 URL 參數中傳遞。
 - 登入狀態由 Next.js `HttpOnly Cookie` 管理。
 - `SESSION_MAX_AGE_SECONDS` 用於伺服器端驗證 session payload 的最長有效時間。
- **請求授權驗證** :
 - 瀏覽器呼叫同源 `/api/*` 時自動帶上 cookie。
 - Next.js route handler 從 cookie 解析並解密驗證 session，確認當前使用者身分。
 - 所有的資料庫查詢與權限限制 (例如資料夾 ACL) 皆在 Next.js 伺服器端根據解密後的 session 資訊直接執行。
- **前端路由機制** :
 - 前端畫面由 Next.js App Router 承載。
 - 切換資料夾、登入狀態還原與頁面重新整理，均應以 Next.js 應用程式狀態與同源 API 回應為準。
 - `GET /api/session` 屬於登入狀態探測路由。未登入或 session cookie 不存在時，回傳 `success: false` 與空資料，不視為 API 例外錯誤。

6.5. Next.js Route Handler 路由與處理服務清單

瀏覽器端呼叫	Next.js Route Handler	伺服器端核心服務與邏輯 (<code>lib/server/</code>)
<code>POST /api/login</code>	<code>app/api/login/route.ts</code>	透過 <code>auth.ts</code> 驗證帳密並寫入 session cookie
<code>POST /api/logout</code>	<code>app/api/logout/route.ts</code>	清除 session cookie
<code>GET /api/session</code>	<code>app/api/session/route.ts</code>	讀取 session cookie 並回傳登入狀態與身分

瀏覽器端呼叫	Next.js Route Handler	伺服器端核心服務與邏輯 (lib/server/)
GET /api/test	app/api/test/route.ts	進行伺服器端資料庫連線測試
GET /api/folders	app/api/folders/route.ts	透過 folderService.ts 查詢資料夾樹與 ACL
POST /api/folders	app/api/folders/route.ts	透過 folderService.ts 建立新資料夾
PUT /api/folders	app/api/folders/route.ts	透過 folderService.ts 重新命名或移動資料夾
DELETE /api/folders	app/api/folders/route.ts	透過 folderService.ts 封存或刪除空資料夾
GET /api/folders/acl	app/api/folders/acl/route.ts	透過 folderService.ts 查詢資料夾有效 ACL 規則
POST /api/folders/acl	app/api/folders/acl/route.ts	透過 folderService.ts 新增或修改資料夾 ACL 授權
GET /api/employee?uid=...	app/api/employee/route.ts	透過 employeeService.ts 以 UID 精準檢索員工資料
GET /api/departments	app/api/departments/route.ts	透過 employeeService.ts 查詢所有部門
GET /api/documents?folder_id=...	app/api/documents/route.ts	透過 documentService.ts 查詢指定資料夾下之有效文件
POST /api/documents	app/api/documents/route.ts	透過 documentService.ts 建立新文件、上傳新版本或修訂

瀏覽器端呼叫	Next.js Route Handler	伺服器端核心服務與邏輯 (lib/server/)
GET /api/documents/download?version_id=...	app/api/documents/download/route.ts	透過 documentService.ts 與 fileStorage.ts 讀取實體檔案並傳送下載串流
GET /api/documents/preview?version_id=...	app/api/documents/preview/route.ts	透過 documentService.ts 與 fileStorage.ts 取得預覽串流，PDF 檔案則動態加入浮水印

DELETE /api/folders 透過 JSON body 的 `action` 欄位區分資料夾生命週期操作：`action = "archive"` 代表封存資料夾；`action = "delete"` 代表刪除（作廢）空資料夾。若未提供 `action`，後端為相容既有呼叫，視為封存資料夾。

6.6. 檔案下載、PDF 預覽與 HTML 預覽處理規格

檔案下載、PDF 預覽與 HTML 預覽由 Next.js Route Handler 直接讀取磁碟實體檔案並處理，向瀏覽器輸出 binary stream 與必要 response header。

- **非 PDF、非 HTML 文件下載：**
 - 瀏覽器呼叫 GET /api/documents/download?version_id=...。
 - Route Handler 呼叫 documentService.downloadDocument 取得磁碟實體檔案與元資料。
 - Next.js 必須保留並設定 content-type、content-disposition（含原始檔名）、content-length、cache-control 等檔案回應 header。
- **PDF 文件預覽：**
 - 瀏覽器呼叫 GET /api/documents/preview?version_id=...。
 - Route Handler 透過 documentService.ts 與 fileStorage.ts 取得 PDF 原始檔，並呼叫 pdfWatermark.ts，使用 pdf-lib 與 @pdf-lib/fontkit 在伺服器端逐頁套用浮水印。
 - 加入浮水印後，Route Handler 必須重新設定正確的 content-length 與 content-type: application/pdf，再將處理後內容回傳瀏覽器。
 - PDF 預覽仍須遵守第 4 節規則：一般使用者僅可線上預覽，不提供 PDF 正式原檔下載。
- **HTML 文件格式預覽：**
 - 瀏覽器呼叫 GET /api/documents/preview?version_id=...。
 - Route Handler 直接以 text/html 回應檔案內容。
 - 瀏覽器端以開新視窗方式承載 .html、.htm、.mhtml 預覽內容。
- **禁止事項：**
 - 不得將敏感之資料庫密碼或密鑰洩漏至瀏覽器端。
 - 不得在客戶端暴露檔案伺服器的實體儲存絕對路徑（如磁碟目錄路徑）。

6.7. 使用者即時檢索 (Lazy Lookup，即時檢索) 規格

為避免使用者人數過大時造成網路頻寬與資料庫效能耗損，本系統全面禁止一次撈出所有使用者資料的行為。

- **API 限制：**後端 /employee API 僅限單一使用者 UID 精準檢索。

- 檢索端點：GET /api/employee?uid=<使用者代碼>。
- 空參數處理：若未傳入 uid 參數，系統不得執行完整使用者清單查詢。
- 前端輸入與防呆：
 - 動態多列輸入框 (Dynamic Row Inputs)：在指派「授權特定同仁」時，使用者以手動逐列輸入使用者代碼，並捨棄傳統的「+ 新增列」按鈕以提升操作流暢度。當輸入框失去焦點 (Blur) 時，系統將自動以非同步方式進行姓名匹配。
 - 自動追加與聚焦 (Auto-append & Focus)：代碼輸入完成並成功檢索出姓名後，系統會自動於下方追加一列新的空白輸入列，並將游標焦點 (Focus) 自動轉移至新欄位，讓使用者能雙手不離鍵盤連續輸入多位同仁代碼。
 - 視覺反饋：若檢索成功，以綠色粗體顯示姓名並標記該列有效；若查無此人，以紅色字樣顯示「查無此使用者」並標記該列無效。
 - 嚴格防呆與強制對話框：
 - 若至少一列輸入代碼但未成功檢索姓名（無效代碼），則系統會顯示紅字警告阻擋存檔，且「確定」按鈕會被禁用。
 - 權限設定等重要對話框 (Modal) 採用強制遮罩 (closeOnOverlayClick=false)，禁止使用者因誤觸背景而關閉視窗導致設定資料遺失。

6.8. 輸入畫面、鍵盤導航與必填驗證規格

- 統一顯示方式：所有輸入畫面使用共用 Modal 元件顯示；工具列搜尋框不屬於輸入畫面，不適用此規則。
- 開啟時焦點：Modal 開啟後，焦點自動位於第一個可見且可編輯的輸入框。
- Enter 導航：
 - 在非最後一個可編輯輸入框按下 Enter，系統阻止預設提交，並將焦點移至下一個可編輯輸入框。
 - 僅在最後一個可編輯輸入框按下 Enter 時，才觸發 Modal 底部操作區中第一個符合 .btn-primary 或 .btn-danger 且未停用的主要按鈕。
 - 中文輸入法組字期間的 Enter 不得觸發欄位導航或提交。
 - 隱藏、唯讀、停用、核取方塊、選項按鈕及檔案欄位不列入鍵盤導航順序。
- 必填欄位驗證：
 - 使用者提交輸入畫面時，若必填欄位未填寫，系統必須顯示明確的錯誤訊息框，不得僅以停用主要按鈕或無訊息返回阻擋操作。
 - 使用者關閉錯誤訊息框後，焦點必須回到第一個未通過驗證的必填欄位；檔案必填欄位則聚焦至對應的檔案選擇按鈕。

7. 版本控管與自動紀錄腳本

本專案使用 Git (版本控管工具) 保存本機版本，並以 GitHub (遠端程式碼代管服務) 作為遠端備份位置。根目錄提供兩個批次檔，分別處理「本機 commit (提交)」與「推送 GitHub (遠端程式碼代管服務)」。

7.1. 排除配置 (.gitignore)

- 根目錄 .gitignore 排除了 dms/node_modules/、dms/dist/、dms/.next/、_deploy/、Delphi 編譯輸出 dmsapi/Win64/、__history/、__recovery/、暫存檔與日誌等不應進入版本庫的內容。
- 若新增工具或產物目錄，應先確認是否需要加入 .gitignore，避免把大型依賴、編譯產物或本機設定提交進 Git (版本控管工具)。

7.2. 本機 Git 備份 (_git.bat)

用途：將目前專案變更保存成本機 Git commit (提交)，不會推送到 GitHub (遠端程式碼代管服務)。

執行方式：

```
_git.bat
```

可指定 commit (提交) 訊息：

```
_git.bat "修正登入與資料夾權限"
```

流程：

- 檢查 Git (版本控管工具) 是否可用。
- 若目前資料夾尚未初始化 Git (版本控管工具)，會自動執行 `git init`。
- 若尚未設定 Git (版本控管工具) 使用者資訊，會設定本專案的本機預設值。
- 顯示目前異動檔案。
- 有變更時執行 `git add -A` 與 `git commit`。
- 沒有變更時不建立空 commit (提交)。

適用時機：完成一段功能修改、修 bug、調整文件或批次檔後，先用此腳本建立本機還原點。

7.3. GitHub 遠端備份 (`_github.bat`)

用途：將本機 `main` 分支推送到 GitHub repository (GitHub 儲存庫)。實際遠端位置以根目錄 Git remote 設定為準。

執行方式：

```
_github.bat
```

可指定自動本機備份的 commit (提交) 訊息：

```
_github.bat "完成資料夾 ACL 調整"
```

流程：

- 檢查目前是否為 Git repository (Git 儲存庫)。
- 檢查目前分支與 `origin` remote (遠端來源)。
- 若偵測到尚未提交的檔案變更，會自動先呼叫 `_git.bat` 建立本機 commit (提交)。
- `_git.bat` 成功後，重新確認工作目錄乾淨。
- 顯示尚未推送到 GitHub (遠端程式碼代管服務) 的 commit (提交)。
- 執行 `git push -u origin <目前分支>`。

若 `_git.bat` 失敗、工作目錄仍有未提交變更、或 GitHub (遠端程式碼代管服務) 權限/網路驗證失敗，腳本會中止推送。

7.4. 建議使用順序

只想先保存本機版本時，執行：

```
_git.bat "本次修改摘要"
```

要同時保存本機並同步 GitHub (遠端程式碼代管服務) 時，直接執行：

```
_github.bat "本次修改摘要"
```

`_github.bat` 已會自動處理本機 commit (提交)，因此日常同步遠端時可直接使用 `_github.bat`。